Dynamic Social Optimum: Sioux Falls Network

COMP3821

1 Definition and reduction to path assignments

The simulator uses linear edge costs $c_e(f) = a_e + b_e f$ and the total social cost

$$SC = \sum_t \sum_{e \in E} f_e(t)\, c_e\big(f_e(t)\big),$$

where $f_e(t)$ is the number of agents that traverse e during timestep t. The *dynamic social optimum* is the lowest value of this expression over every feasible way of routing the agents:

$$\boxed{SC_{\text{opt}} = \min_{\text{feasible routing decisions}} \sum_t \sum_{e \in E} f_e(t)\, c_e\big(f_e(t)\big)}$$

Feasible routings. The movement rules of the specification (Section 3.1) and the simulator fix what "feasible" means:

1. all N agents are released at $t = 0$ (none of these scenarios uses cascades);
2. at every timestep each unfinished agent traverses exactly one edge, so no agent can wait;
3. with `forward_only` (the default), an agent never moves to a node with smaller hop distance from its origin and never revisits a node.

Lemma 1 (Routing = path assignment)**.** *Let $\mathcal{P}_{od}$ be the simple paths from o to d that obey rule 3. Every feasible routing gives each agent a path $P_i \in \mathcal{P}_{o_i d_i}$, and agent i traverses the k-th edge of P_i at timestep $t = k - 1$. Every assignment of such paths can be realised. Therefore*

$$SC = \sum_{e \in E} \sum_{t \geq 0} X_{e,t}\,\big(a_e + b_e X_{e,t}\big), \qquad X_{e,t} = \#\{\, i : e \text{ is the } (t+1)\text{-th edge of } P_i \,\},$$

and SC_{opt} is the minimum of this over all path assignments.

Proof. Rule 2 means an agent's position at time t is the end of the t-th edge it has taken. Rule 3 means the edges it takes form a simple path in the allowed subgraph. So $f_e(t) = X_{e,t}$. Conversely, a central planner can tell each agent which next edge to commit to at each step. The outcome depends only on the paths, so every path assignment is achievable. □

So the dynamic problem is a *static* problem on the *time-expanded* resources (e, t). Agents on the same edge only slow each other down if they cross it in the same step.

Lemma 2 (When the dynamic and static optima coincide)**.** *Suppose every edge e sits at the same position $k(e)$ on every feasible path that contains it. Then $X_{e,t} = x_e\,\mathbf{1}[t = k(e) - 1]$, where x_e is the total number of agents using e. So $SC = \sum_e x_e\, c_e(x_e)$, which is the classical static social-optimum objective (Beckmann; Roughgarden) restricted to integer flows.*

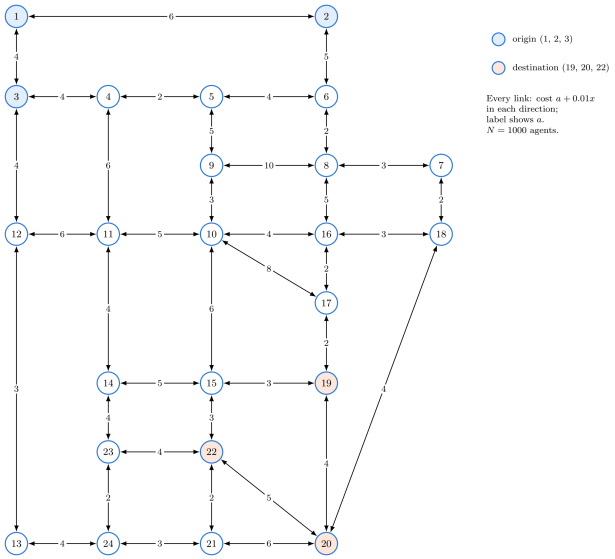
Integer versus continuous. Agents are indivisible, so SC_{opt} is an integer program. Relaxing the path counts to non-negative reals gives a convex quadratic program. Its minimum is a lower bound on SC_{opt} and equals the limit of SC_{opt}/N for large N. Both are reported below. Because c_e is linear, the cost one extra agent adds to resource (e,t) is

$$\Delta^+_{e,t} = (X+1)\,c_e(X+1) - X\,c_e(X) = a_e + b_e(2X+1), \qquad X = X_{e,t},$$

and in the continuous relaxation the marginal cost is $\frac{d}{dX}\big[X c_e(X)\big] = a_e + 2b_eX$.

Scaling with N. Every small scenario sets $b_e = \beta_e/N$ with β_e fixed, so the per-agent optimum SC_{opt}/N is almost the same for $N = 20, 50, 80$.

2 Network and demand



The scenario `scenarios/sioux_falls` has 24 nodes and 76 directed edges. Every edge is linear, with $a_e \in \{2,3,4,5,6,8,10\}$ and the same slope $b_e = 0.01$. There are $N = 1000$ agents. Each agent draws its (o,d) pair uniformly from sources $\{1,2,3\}$ and destinations $\{19,20,22\}$, using the run's seed. The demand q_{od} therefore depends on the seed. The main results use the config seed (42), and Table 3 repeats the calculation for the 30 sweep seeds. `forward_only` is on, and $\mathcal{P}_{od}$ (the allowed simple paths) has between 23 and 94 paths per OD pair, 404 in total.

3 Derivation

Timing matters here. Paths have 5–12 edges, and the same edge appears at different positions on different paths. For example, $2 \to 6$ is the first edge of $2 \to 6 \to 8 \to \cdots$ but the second edge of $1 \to 2 \to 6 \to \cdots$. Lemma 2 does not apply, so the optimum must be taken over the time-expanded resources (e,t) of Lemma 1.

Problem. By Lemma 1,

$$SC_{\text{opt}} = \min_{n} \sum_{(e,t)} \big(a_e X_{e,t} + b_e X_{e,t}^2\big)$$

$$\text{s.t.} \quad X_{e,t} = \sum_{od} \sum_{P \in \mathcal{P}_{od}:\,(e,t)\in P} n_P, \qquad \sum_{P\in\mathcal{P}_{od}} n_P = q_{od}, \qquad n_P \in \mathbb{Z}_{\geq 0},$$

where $(e,t) \in P$ means e is the $(t+1)$-th edge of P. This is a separable convex integer program with 9 commodities and 404 path variables. Unlike the small networks it has no closed form, so we solve it numerically and bracket it between two certified bounds.

Step 1: lower bound from the continuous relaxation. Relax $n_P \geq 0$ to real values. For a flow y, the *marginal path cost* is

$$M_P(y) = \sum_{(e,t)\in P} \big(a_e + 2b_e X_{e,t}(y)\big).$$

The relaxation is convex, and y is optimal exactly when, for each OD pair, every used path has the same marginal cost λ_{od} and no path has a smaller one. We solved it with path-based gradient projection (shift flow from each dearer path to the cheapest marginal path by a Newton step). Every convex function lies above its tangent plane, so for *any* feasible y and any integer routing n,

$$SC(n) \;\geq\; SC(y) + \nabla SC(y)\cdot(n-y) \;\geq\; SC(y) + \sum_{od} q_{od} \min_{P\in\mathcal{P}_{od}} M_P(y) \;-\; \sum_P y_P M_P(y) \;=:\; \mathrm{LB}(y).$$

This is the Frank–Wolfe dual bound. It is valid whether or not y is exactly optimal. At the solution found, the two sides agree to 10^{-9}, giving LB = 26684.8126 for seed 42.

Step 2: upper bound from an integer routing. We round the continuous solution within each OD pair (largest remainder), then repeatedly move a single agent to another path while that lowers SC. A move from P to Q changes the cost by $\sum_{(e,t)\in Q\setminus P}[a_e + b_e(2X_{e,t}+1)] - \sum_{(e,t)\in P\setminus Q}[a_e + b_e(2X_{e,t}-1)]$. At the end no single-agent switch helps. The resulting routing (Table 2) costs 26684.91.

Conclusion. $26684.81 \leq SC_{\mathrm{opt}} \leq 26684.91$. The integer routing is within 0.1 (3.7×10^{-6} relative) of the true optimum, which is far below anything the simulations can resolve. We report the integer value as SC_{opt}.

4 Result (config seed 42)

$SC_{\mathrm{opt}} = 26684.91$ (26.685 per agent), $\qquad 26684.81 \leq SC_{\mathrm{opt}} \leq 26684.91$

OD pair	Agents	Feasible paths	Paths used	Min. $\sum a_e$	Mean cost at optimum
1→19	99	40	4	22	28.885
1→20	118	40	3	22	29.367
1→22	97	40	2	20	26.646
2→19	124	23	1	16	24.890
2→20	122	23	1	16	24.830
2→22	93	94	3	21	30.454
3→19	114	48	8	21	26.198
3→20	121	48	3	20	27.047
3→22	112	48	6	16	22.931
All	1000				26.685

Table 1: Demand and optimal routing summary, seed 42.

What the optimum looks like.

- Every agent arrives within 8 steps. The number of agents still moving at steps $0, \ldots, 7$ is $1000, 1000, 1000, 1000, 1000, 523, 198, 24$.
- Of the cost, 20437.0 is base travel cost ($\sum a_e X_{e,t}$) and 6247.9 is congestion ($\sum b_e X_{e,t}^2$).
- The optimum *staggers* agents in time on shared corridors. For example, edge $2 \to 6$ carries 271, 139 and 17 agents at $t = 0, 1, 2$, and $13 \to 24$ carries 153, 113 and 68 at $t = 2, 3, 4$. Agents from origin 1 or 3 who go via node 2 reach the corridor one step behind the origin-2 agents and add nothing to their congestion. A static model cannot do this.
- A few agents use paths up to 7 units longer in base cost, such as $1 \to 3 \to 4 \to 5 \to 9 \to 10 \to 16 \to 18 \to 20$ (7 agents), because doing so relieves a congested resource for many others.

Table 2: An optimal integer routing for the config seed (42), per OD pair. "Realised cost" is what one agent on that path pays.

OD	Path	Agents	$\sum_{e \in P} a_e$	Realised cost
1→19	1-2-6-8-16-17-19	44	22	27.75
	1-3-4-11-14-15-19	35	26	29.77
	1-3-4-5-9-10-16-17-19	17	26	29.76
	1-3-4-5-9-10-15-19	3	27	30.24
1→20	1-2-6-8-7-18-20	95	22	29.02
	1-3-12-13-24-21-20	16	24	30.03
	1-3-4-5-9-10-16-18-20	7	29	32.56
1→22	1-3-12-13-24-21-22	57	20	26.44
	1-3-12-13-24-23-22	40	21	26.94
2→19	2-6-8-16-17-19	124	16	24.89
2→20	2-6-8-7-18-20	122	16	24.83
2→22	2-1-3-12-13-24-21-22	46	26	30.32
	2-6-5-9-10-15-22	25	26	30.36
	2-1-3-12-13-24-23-22	22	27	30.84
3→19	3-4-11-14-15-19	56	22	26.09
	3-4-5-9-10-16-17-19	20	22	26.10
	3-4-5-6-8-16-17-19	12	21	25.62
	3-12-11-10-16-17-19	10	23	26.60
	3-12-11-10-15-19	9	24	27.10
	3-4-11-10-16-17-19	3	23	26.59
	3-12-11-14-15-19	3	22	26.10
	3-1-2-6-8-16-17-19	1	26	28.13
3→20	3-4-5-6-8-7-18-20	54	21	26.93
	3-12-13-24-21-20	51	20	26.42
	3-1-2-6-8-7-18-20	16	26	29.44
3→22	3-12-13-24-21-22	52	16	22.43
	3-12-13-24-23-22	50	17	22.95
	3-12-11-14-23-22	6	22	25.44
	3-4-11-14-15-22	2	22	25.44
	3-4-11-14-23-22	1	22	25.43

Table 2: An optimal integer routing for the config seed (42), per OD pair. "Realised cost" is what one agent on that path pays.

OD	Path	Agents	$\sum_{e\in P} a_e$	Realised cost
	3-12-11-14-15-22	1	22	25.45

5 Comparison

All values below are for seed 42 (the config seed).

Quantity	Total	Ratio to SC_{opt}
Dynamic social optimum SC_{opt} (this document)	26684.91	1.000
Static (literature) optimum routing, costed in the simulator	28497.98	1.068
Simulator, $\alpha = 0.25$ (all agents)	29089.57	1.090
Simulator, $\alpha = 0$ (all agents)	29484.03	1.105
Simulator, $\alpha = 1$ (all agents)	31137.72	1.167
Static (literature) social optimum value, $\sum_e x_e c_e(x_e)$[1]	32612.46	1.222

The static optimum charges every agent on an edge for everyone who *ever* uses it, so its value (32612) is not attainable or comparable in the step-based simulator. Its routing, run through the simulator's cost, gives 28498, still 6.8% above the dynamic optimum because it ignores timing. The correct benchmark for the simulation results is SC_{opt}.

6 Sensitivity to the demand sample

The sweep script (`symmetric_sweep.py`) uses seeds $3821 + t$ for trials $t = 0, \ldots, 29$, and each seed draws a different OD split. SC_{opt} therefore varies slightly between trials. For each seed, the integer solution and the lower bound were found as above:

Seed	Lower bound	SC_{opt} (integer)	Relative gap	Per agent
42	26684.81	26684.91	3.6e-06	26.6849
3821	26720.78	26720.88	3.6e-06	26.7209
3822	26712.38	26712.49	4.3e-06	26.7125
3823	26768.91	26768.99	2.9e-06	26.7690
3824	26613.46	26613.56	3.6e-06	26.6136
3825	26801.33	26801.43	3.8e-06	26.8014
3826	26782.09	26782.20	3.9e-06	26.7822
3827	26853.54	26853.64	3.6e-06	26.8536
3828	26740.20	26740.29	3.3e-06	26.7403
3829	26684.65	26684.73	3.1e-06	26.6847
3830	26815.12	26815.20	2.9e-06	26.8152
3831	26814.62	26814.72	3.6e-06	26.8147
3832	26709.66	26709.76	3.7e-06	26.7098
3833	26677.02	26677.10	2.8e-06	26.6771
3834	26810.53	26810.59	2.3e-06	26.8106
3835	26725.61	26725.69	2.9e-06	26.7257
3836	26731.19	26731.25	2.1e-06	26.7312

[1] Matches `src/benchmarks/frank_wolfe.py --demand sampled`, whose continuous social optimum is 32612.42.

	Seed	Lower bound	SC_{opt} (integer)	Relative gap	Per agent
	3837	26725.55	26725.62	2.6e-06	26.7256
	3838	26795.45	26795.55	3.7e-06	26.7955
	3839	26758.26	26758.34	3.1e-06	26.7583
	3840	26787.06	26787.14	3.1e-06	26.7871
	3841	26772.81	26772.92	3.9e-06	26.7729
	3842	26675.15	26675.28	4.7e-06	26.6753
	3843	26635.06	26635.15	3.4e-06	26.6352
	3844	26770.88	26770.99	4.1e-06	26.7710
	3845	26770.75	26770.80	1.9e-06	26.7708
	3846	26774.52	26774.63	4.1e-06	26.7746
	3847	26679.80	26679.89	3.3e-06	26.6799
	3848	26858.29	26858.41	4.3e-06	26.8584
	3849	26692.56	26692.65	3.5e-06	26.6927
	3850	26617.04	26617.13	3.3e-06	26.6171
Seeds 3821–3850			mean 26742.57	sd 64.55	26.7426
			min 26613.56	max 26858.41	

Table 3: Optimum for the config seed (42) and the 30 seeds used by `symmetric_sweep.py`.

Across the sweep seeds, $SC_{\text{opt}}/N = 26.743 \pm 0.065$ (mean $\pm$ sd). To compare a trial with its own optimum, use that seed's row. To draw a single reference line, use the mean, 26.743 per agent.